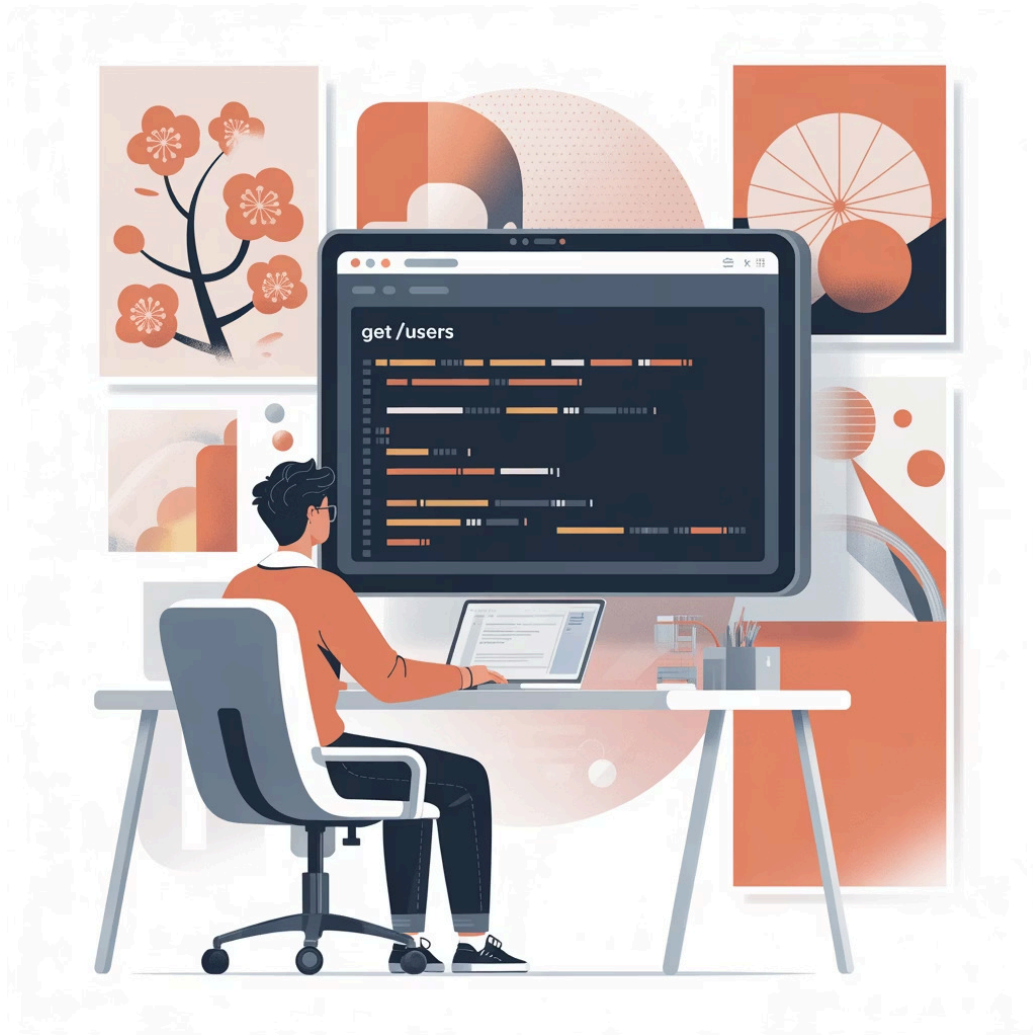


# Métodos HTTP en Spring Boot: GET, POST, PUT, DELETE

Java con Spring Boot de 0 a Pro  
Video #3 - APIs REST con Spring Boot



# ¿Qué aprenderás hoy?



En este video implementaremos los cuatro métodos HTTP principales en Spring Boot y aprenderás a probarlos efectivamente.

## Agenda del video:

- Repaso rápido de métodos HTTP
- Implementación de endpoints GET, POST, PUT y DELETE
- Pruebas prácticas en Postman
- Consejos para desarrollo efectivo

# Recordatorio: Los 4 Métodos HTTP Esenciales

## GET

### Leer datos

Recupera información del servidor sin modificarla. Es seguro y se puede repetir sin efectos secundarios.

## POST

### Crear recursos

Envía datos al servidor para crear un nuevo recurso. Cada llamada puede tener efectos diferentes.

## PUT

### Actualizar completo

Reemplaza completamente un recurso existente con los datos enviados en la petición.

## DELETE

### Eliminar recurso

Elimina un recurso específico del servidor de forma permanente.

# Estructura de Controlador con Métodos HTTP

Veamos cómo implementar los cuatro métodos HTTP en un controlador de Spring Boot. Cada anotación corresponde a un método HTTP específico.

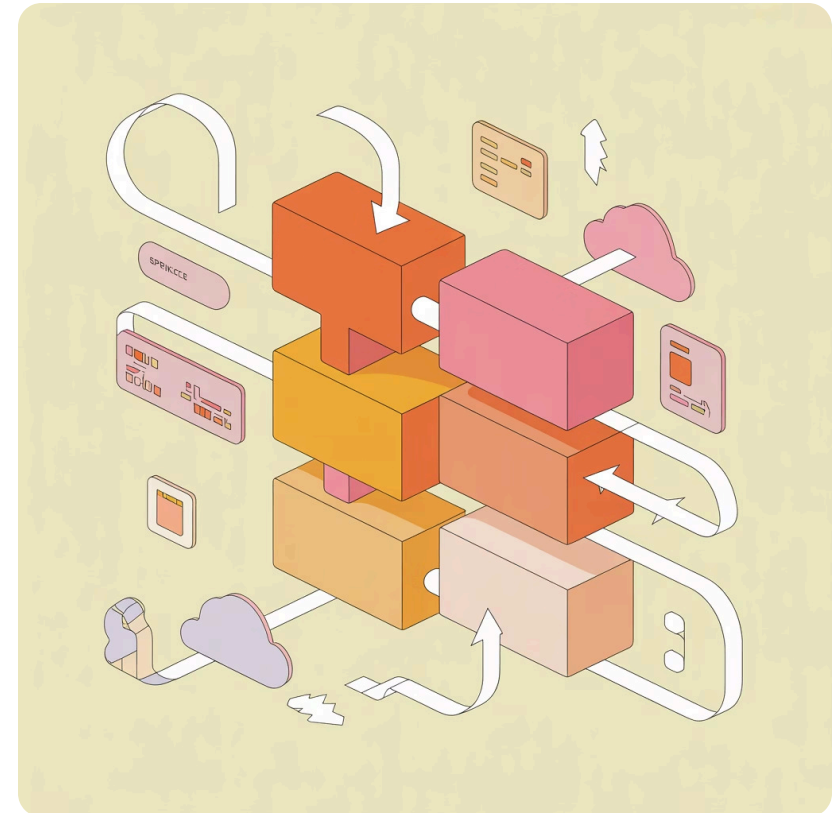
```
@RestController
@RequestMapping("/api/ejemplo")
public class EjemploController {

    @GetMapping
    public String obtenerDatos() {
        return "Datos obtenidos correctamente";
    }

    @PostMapping
    public String crearRecurso() {
        return "Recurso creado exitosamente";
    }

    @PutMapping("/{id}")
    public String actualizarRecurso(@PathVariable Long id) {
        return "Recurso " + id + " actualizado";
    }

    @DeleteMapping("/{id}")
    public String eliminarRecurso(@PathVariable Long id) {
        return "Recurso " + id + " eliminado";
    }
}
```



**Tip:** Cada anotación (@GetMapping, @PostMapping, etc.) es una forma abreviada de @RequestMapping con el método HTTP especificado.

# Endpoints de Ejemplo Implementados

01

---

## GET /api/ejemplo

Devuelve un mensaje de confirmación simple. Útil para verificar que el servicio está funcionando correctamente.

03

---

## PUT /api/ejemplo/{id}

Actualiza un recurso específico identificado por su ID. El parámetro {id} se captura con @PathVariable.

02

---

## POST /api/ejemplo

Simula la creación de un nuevo recurso. En aplicaciones reales, aquí recibirías datos en el cuerpo de la petición.

04

---

## DELETE /api/ejemplo/{id}

Elimina el recurso especificado por el ID. Confirma la eliminación con un mensaje de respuesta.

# Checklist para la Demostración



## Antes de empezar:

- Servidor Spring Boot ejecutándose
- Postman abierto y configurado
- Puerto 8080 disponible

## Probaremos en orden:

1. GET → `http://localhost:8080/api/ejemplo`
2. POST → `http://localhost:8080/api/ejemplo`
3. PUT → `http://localhost:8080/api/ejemplo/1`
4. DELETE → `http://localhost:8080/api/ejemplo/1`

⚠ Recuerda cambiar el método HTTP en Postman para cada prueba. Por defecto viene configurado como GET.

# [Demostración en vivo con código]

A continuación veremos la implementación paso a paso de cada endpoint y las pruebas en Postman para verificar su funcionamiento correcto.

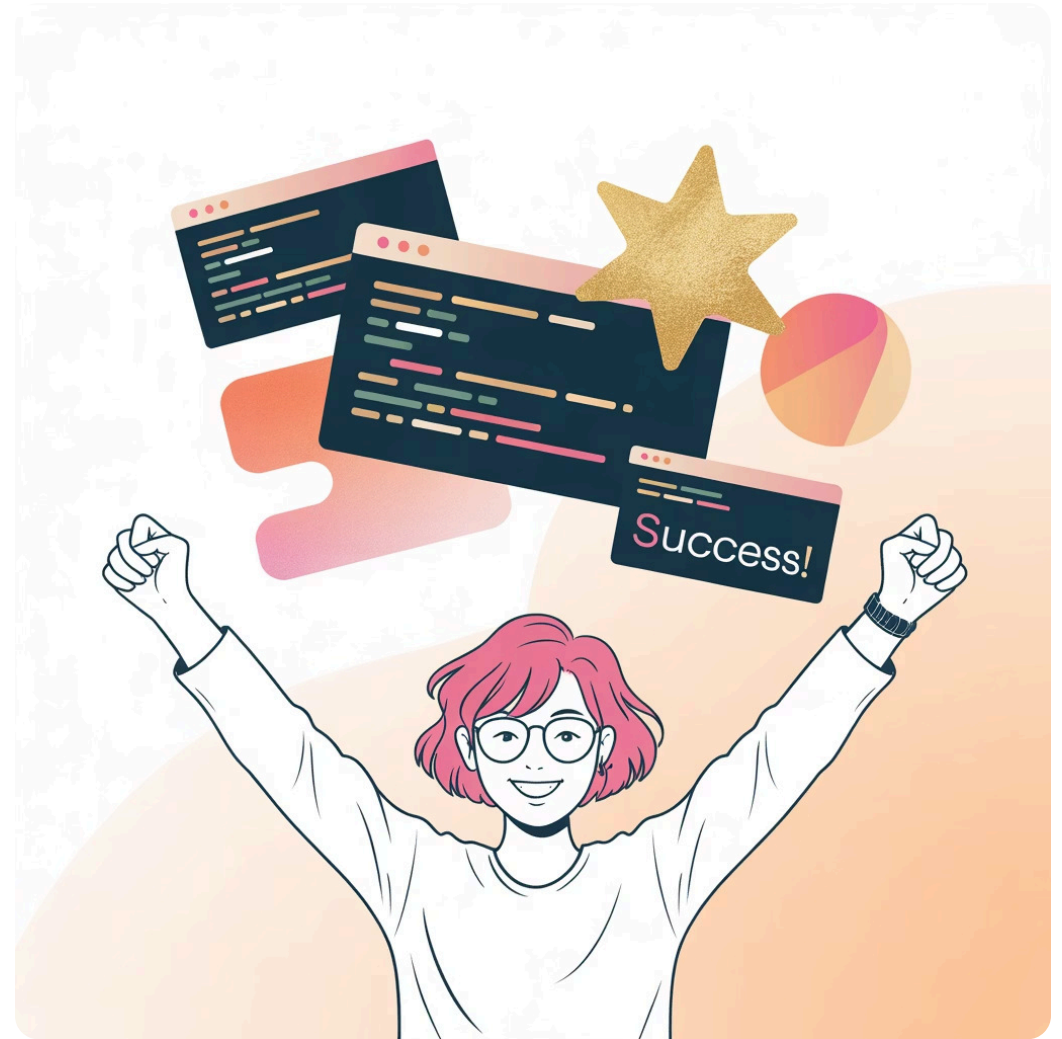


# Resumen y Próximos Pasos

## Lo que aprendiste:

- Implementar los 4 métodos HTTP principales
- Usar `@PathVariable` para capturar parámetros
- Probar endpoints con Postman
- Estructura básica de controladores REST

✅ ¡Felicidades! Ya puedes crear APIs REST básicas con los métodos HTTP esenciales.



## En el próximo video:

Aprenderemos a manejar el cuerpo de las peticiones HTTP y trabajar con datos JSON en nuestros endpoints POST y PUT.

¡Nos vemos en el siguiente video de la serie!